JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicInteger

java.lang.Object
    java.lang.Number
        java.util.concurrent.atomic.AtomicInteger

All Implemented Interfaces:

Serializable

```
public class AtomicInteger
extends Number
implements Serializable
```

An `int` value that may be updated atomically. See the `VarHandle` specification for descriptions of the properties of atomic accesses. An `AtomicInteger` is used in applications such as atomically incremented counters, and cannot be used as a replacement for an `Integer`. However, this class does extend `Number` to allow uniform access by tools and utilities that deal with numerically-based classes.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicInteger()`	Creates a new AtomicInteger with initial value `0`.
`AtomicInteger(int initialValue)`	Creates a new AtomicInteger with the given initial value.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`final int`	`accumulateAndGet(int x, IntBinaryOperator accumu`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with

		the results of applying the given function to the current and given values, returning the updated value.
final int	**addAndGet**(int delta)	Atomically adds the given value to the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final int	**compareAndExchange**(int expectedValue, int newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchange(Object...
final int	**compareAndExchangeAcquir**(int expectedValue, int newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeAcquire(Obje
final int	**compareAndExchangeReleas**(int expectedValue, int newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeRelease(Obje

final boolean	**compareAndSet**(int expectedValue, int newValue)	Atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).
final int	**decrementAndGet**()	Atomically decrements the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
double	**doubleValue**()	Returns the current value of this AtomicInteger as a double after a widening primitive conversion, with memory effects as specified by VarHandle.getVolatile(Object...).
float	**floatValue**()	Returns the current value of this AtomicInteger as a float after a widening primitive conversion, with memory effects as specified by VarHandle.getVolatile(Object...).
final int	**get**()	Returns the current value, with memory effects as specified by VarHandle.getVolatile(Object...).
final int	**getAcquire**()	Returns the current value, with memory effects as specified by VarHandle.getAcquire(Object...).
final int	**getAndAccumulate**(int x, IntBinaryOperator accumu	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...))

		the current value with the results of applying the given function to the current and given values, returning the previous value.
final int	**getAndAdd**(int delta)	Atomically adds the given value to the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final int	**getAndDecrement**()	Atomically decrements the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final int	**getAndIncrement**()	Atomically increments the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final int	**getAndSet**(int newValue)	Atomically sets the value to newValue and returns the old value, with memory effects as specified by VarHandle.getAndSet(Object...).
final int	**getAndUpdate**(IntUnaryOperator update	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying the given function,

		returning the previous value.
final int	**getOpaque**()	Returns the current value, with memory effects as specified by VarHandle.getOpaque(Object...).
final int	**getPlain**()	Returns the current value, with memory semantics of reading as if the variable was declared non-volatile.
final int	**incrementAndGet**()	Atomically increments the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
int	**intValue**()	Returns the current value of this AtomicInteger as an int, with memory effects as specified by VarHandle.getVolatile(Object...).
final void	**lazySet**(int newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
long	**longValue**()	Returns the current value of this AtomicInteger as a long after a widening primitive conversion, with memory effects as

		specified by `VarHandle.getVolatile(Object...)`.
final void	**set**(int newValue)	Sets the value to `newValue`, with memory effects as specified by `VarHandle.setVolatile(Object...)`.
final void	**setOpaque**(int newValue)	Sets the value to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.
final void	**setPlain**(int newValue)	Sets the value to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.
final void	**setRelease**(int newValue)	Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.
`String`	**toString**()	Returns the String representation of the current value.
final int	**updateAndGet**(`IntUnaryOperator` update	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the updated value.
final boolean		**Deprecated.**

	weakCompareAndSet (int expectedValue, int newValue)	This method has plain memory effects but the method name implies volatile memory effects (see methods such as compareAndExchange(int, int) and compareAndSet(int, int)).
final boolean	**weakCompareAndSetAcquire** (int expectedValue, int newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetAcquire(Objec
final boolean	**weakCompareAndSetPlain** (int expectedValue, int newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease** (int expectedValue, int newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatil** (int expectedValue, int newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSet(Object...).

Methods declared in class Number

`byteValue`, `shortValue`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

AtomicInteger

```
public AtomicInteger(int initialValue)
```

Creates a new AtomicInteger with the given initial value.

Parameters:

`initialValue` - the initial value

AtomicInteger

```
public AtomicInteger()
```

Creates a new AtomicInteger with initial value `0`.

Method Details

get

```
public final int get()
```

Returns the current value, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Returns:

the current value

set

```
public final void set(int newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setVolatile(Object...)`.

Parameters:

`newValue` - the new value

lazySet

```
public final void lazySet(int newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

1.6

getAndSet

```
public final int getAndSet(int newValue)
```

Atomically sets the value to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.

Parameters:

`newValue` - the new value

Returns:

the previous value

compareAndSet

```
public final boolean compareAndSet(int expectedValue,
                                   int newValue)
```

Atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public final boolean weakCompareAndSet(int expectedValue,
                                       int newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, int)` and `compareAndSet(int, int)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(int, int)` be used instead.

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(int, int)`

weakCompareAndSetPlain

```
public final boolean weakCompareAndSetPlain(int expectedValue,
                                            int newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

getAndIncrement

```
public final int getAndIncrement()
```

Atomically increments the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(1)`.

Returns:

the previous value

getAndDecrement

```
public final int getAndDecrement()
```

Atomically decrements the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(-1)`.

Returns:

the previous value

getAndAdd

```
public final int getAndAdd(int delta)
```

Atomically adds the given value to the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`delta` - the value to add

Returns:

the previous value

incrementAndGet

```
public final int incrementAndGet()
```

Atomically increments the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(1)`.

Returns:

the updated value

decrementAndGet

```
public final int decrementAndGet()
```

Atomically decrements the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(-1)`.

Returns:

the updated value

addAndGet

```
public final int addAndGet(int delta)
```

Atomically adds the given value to the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`delta` - the value to add

Returns:

the updated value

getAndUpdate

```
public final int getAndUpdate(IntUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final int updateAndGet(IntUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction` - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final int getAndAccumulate
(int x,
 IntBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final int accumulateAndGet
(int x,
 IntBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

intValue

```
public int intValue()
```

Returns the current value of this `AtomicInteger` as an `int`, with memory effects as specified by `VarHandle.getVolatile(Object...)`. Equivalent to `get()`.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `int`.

longValue

```
public long longValue()
```

Returns the current value of this `AtomicInteger` as a `long` after a widening primitive conversion, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `long`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

floatValue

```
public float floatValue()
```

Returns the current value of this `AtomicInteger` as a `float` after a widening primitive conversion, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `float`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

doubleValue

```
public double doubleValue()
```

Returns the current value of this `AtomicInteger` as a `double` after a widening primitive conversion, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `double`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

getPlain

```
public final int getPlain()
```

Returns the current value, with memory semantics of reading as if the variable was declared non-`volatile`.

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(int newValue)
```

Sets the value to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`newValue` - the new value

Since:

9

getOpaque

```
public final int getOpaque()
```

Returns the current value, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(int newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

getAcquire

```
public final int getAcquire()
```

Returns the current value, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(int newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

compareAndExchange

```
public final int compareAndExchange(int expectedValue,
                                    int newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final int compareAndExchangeAcquire(int expectedValue,
                                           int newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final int compareAndExchangeRelease(int expectedValue,
                                           int newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(int expectedValue,
                                               int newValue)
```

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(int expectedValue,
                                              int newValue)
```

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(int expectedValue,
                                              int newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.